

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Криптография - пишем и взламываем шифры

Всевозможные шифры начали придумывать ещё с древних времён, чтобы в случае перехвата гонца, противник не смог узнать что написано в донесении. Шифры широко применяются как в военном деле, так и для обеспечения безопасной передачи данных, тайны переписки, сохранения конфиденциальной информации на съёмных носителях и т.д.

Рассмотрим несколько простых шифров, которые положили начало развитию более сложных вариантов шифрования.

Шифр Атбаш

Шифр Атбаш является одним из самых лёгких методов шифрования. Создаёт зеркальную версию алфавита, где начало - буква Z, а конец алфавита буква - А.

ABCDEFGHIJKLMNOPQRSTUVWXYZ
ZYXWVUTSRQPONMLKJIHGFEDCBA

Для создания списка букв алфавита верхнего регистра в переменной `alphabet` применим коды символов ASCII в диапазоне **65-91**. В переменной `alpha` продублируем список, и сделаем зеркальное расположение символов с помощью метода `reverse()`.

Далее перебираем в цикле символы из сообщения. Во вложенном цикле перебираем индексы символов. Задаём условие, при котором если введенный символ соответствует символу, который присутствует в алфавите зеркального варианта, то добавляем его в строку. По окончании цикла, выводим получившуюся строку с символами.

```
message = input("Enter a message: ").upper()
alphabet = [chr(x) for x in range(65, 91)]
alpha = list(alphabet)
alpha.reverse()
final = ""
for symbol in message:
    for index_alpha, symbol_alpha in enumerate(alphabet):
        if symbol == symbol_alpha:
            final += alpha[index_alpha]
print("Encrypted message:", final)
```

```
atbash x
Enter a message: Wonderful
Encrypted message: DLMWVIUFO
```

Для расшифровки сообщения всего лишь нужно стандартный алфавит и зеркальный поменять местами в цикле.

```

message = input("Enter a message: ").upper()
alphabet = [chr(x) for x in range(65, 91)]
alpha = list(alphabet)
alpha.reverse()
final = ""
for symbol in message:
    for index_alpha, symbol_alpha in enumerate(alpha):
        if symbol == symbol_alpha:
            final += alphabet[index_alpha]
print("Decrypted message:", final)

```

atbash x

Enter a message: DLMWVIUFO
Decrypted message: WONDERFUL

Шифр Цезаря

Шифр Цезаря заключается в замене каждого символа входной строки на символ, находящийся на несколько позиций левее или правее его в алфавите.

Для всех символов сдвиг один и тот же. Сдвиг циклический, то есть если к последнему символу алфавита применить единичный сдвиг, то он заменится на первый символ, и наоборот.

Пример со сдвигом на 3 символа, то есть $A+3 = D$:

ABCDEFGHIJKLMNOPQRSTUVWXYZ
 DEFGHIJKLMNOPQRSTUVWXYZABC

Сначала напишем выбор опции шифровки/дешифровки. Если опция выбрана 1 или 2, активируем соответствующую функцию, иначе выдаём пользователю сообщение "Enter 1 or 2!".

```

print('1 - encode text\n2 - decode text\n')
action = input("Select action: ")
if action == '1':
    text = input("Enter a message: ")
    s = int(input("Enter shift: "))
    print("Encrypted text: " + encrypt(text, s))
elif action == '2':
    text = input("Enter a message: ")
    s = int(input("Enter shift: "))
    print("Decrypted text: " + decrypt(text, s))
else:
    print("Enter 1 or 2!")

```

Функции шифровки/дешифровки сообщения одинаковые, с разницей лишь в вычитании переменной s (сдвиг символов), а не прибавлении.

```

def encrypt(mes, l):
    result = ""
    for i in range(len(mes)):
        char = mes[i]
        if char.isupper():
            result += chr((ord(char) + l - 65) % 26 + 65)
        else:
            result += chr((ord(char) + l - 97) % 26 + 97)
    return result

def decrypt(mes, l):
    result = ""
    for i in range(len(mes)):
        char = mes[i]
        if char.isupper():
            result += chr((ord(char) - l - 65) % 26 + 65)
        else:
            result += chr((ord(char) - l - 97) % 26 + 97)
    return result

```

Шифрование сообщения производится при помощи таблицы ASCII. В цикле мы перебираем каждый символ, и если символ в верхнем регистре, то прибавляем к нему длину сдвига, и отнимаем 65 (Код буквы **A**). Допустим первая буква в сообщении Z и сдвиг 3, получится $90+3 = 93$, далее $93-65 = 28$ при делении с остатком на 26 (количество букв), получится 2. И прибавив $2 + 65$ получим число 67 (Код буквы **C**).

ord() - Возвращает код символа

chr() - Возвращает символ, соответствующий коду

Шифр пар(замены)

Шифр пар или шифр замены не требует отдельного кода для дешифровки. Особенность данного метода шифрования в том, что есть привязка между двумя символами. То есть, если в тексте попадаете символ A, тогда он заменяется на B ($A \rightarrow B$), но это также действует и в обратную сторону ($B \rightarrow A$).

В самом простейшем виде можно представить пары соседних символов. Сделаем словарь, где ключ будет каким-либо символом, а значение следующим за ним символом.

Если символа не оказалось в словаре, то он игнорируется.


```
keys = {
    'A': 'B', 'C': 'D', 'E': 'F', 'G': 'H', 'I': 'J', 'K': 'L',
    'M': 'N', 'O': 'P', 'Q': 'R', 'S': 'T', 'U': 'V', 'W': 'X',
    'Y': 'Z'}
message = list(input("Enter a message: ").upper())
for symbol in range(len(message)):
    for key in keys:
        if message[symbol] == key:
            message[symbol] = keys[key]
        elif message[symbol] == keys[key]:
            message[symbol] = key
        else:
            pass
print("Encrypted/decrypted message:", "".join(message))
```

two x

Enter a message: **BLABLABLA**

Encrypted/decrypted message: **AKBAKBAKB**

two x

Enter a message: **AKBAKBAKB**

Encrypted/decrypted message: **BLABLABLA**

Process finished with exit code 0

Все эти шифры весьма примитивные, и легко расшифровываются. Кто интересуется написанием действительно взломостойкого шифра, может ознакомиться [с нестандартным вариантом кода](#).